

Technical Assignment

Senior Full-Stack Engineer

Project: Multi-Tenant Real-Time Order & Inventory Management Platform

Project type: Full-Stack / System Design / Architecture

SUBMISSION DEADLINE

Monday, 21 September 2026 — End of Day (11:59 PM IST)

Thank you for your interest in joining Uphead Management Consulting. This assignment is designed to understand how you approach a production-oriented system: the architectural decisions you make, the trade-offs you weigh, and the quality of what you ship. Please read the document fully before you begin, and document your reasoning as you go — we care as much about **how** you build as **what** you build.

1. Project Overview

Build a production-oriented platform where multiple organizations can independently manage:

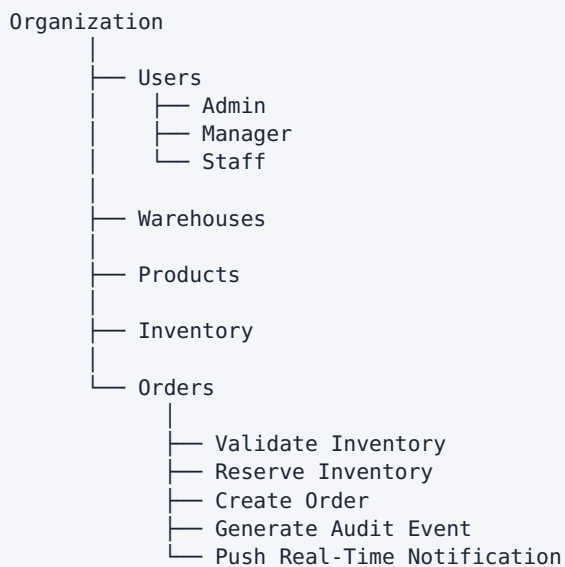
- Products
- Inventory
- Warehouses
- Customers
- Orders
- Users
- Roles and permissions
- Notifications
- Audit history

The system must support multiple organizations while ensuring complete data isolation between tenants. Example organizations could be retail chains, automobile dealerships, distributors, e-commerce businesses, or warehouse operators.

The objective is **not** simply to build CRUD APIs. The assignment should demonstrate your ability to make architectural decisions suitable for a scalable production system.

2. Primary Objective

Design and implement a system capable of handling the following flow:



Multiple organizations must be able to use the same application without accessing each other's information.

3. Functional Requirements

3.1 Authentication

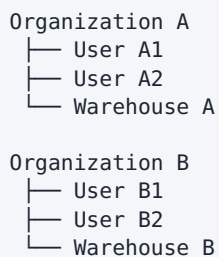
Implement authentication using either JWT access + refresh tokens, OAuth2, or OpenID Connect.

Required functionality: login, logout, refresh token, token expiration, password hashing, and invalid/revoked token handling.

Bonus: multiple active sessions, device/session management, refresh token rotation.

3.2 Multi-Tenancy

Every user belongs to an organization.



A user belonging to Organization A must never be able to access Organization B's products, orders, customers, inventory, users, or reports. You should document how tenant isolation has been implemented.

3.3 Role-Based Access Control

Role	Permissions
Admin	Full organization access
Manager	Manage products, inventory and orders
Staff	Read inventory and process orders

Permissions should be enforced at the backend level. Frontend-only permission checks are not acceptable.

Bonus: implement granular permission-based authorization, e.g.

```
inventory.read
inventory.update
order.create
order.cancel
user.manage
report.view
```

4. Product Management

Each organization can maintain its own product catalog. Required fields:

```
id
organizationId
sku
name
description
price
status
createdAt
updatedAt
```

Features: create product, update product, disable product, product details, paginated product listing, search, filtering, and sorting. SKU should be unique within an organization.

5. Warehouse Management

Organizations should be able to create multiple warehouses.

```
Organization
├── Kolkata Warehouse
├── Mumbai Warehouse
└── Delhi Warehouse
```

Warehouse fields:

```
id
organizationId
name
address
status
createdAt
updatedAt
```

6. Inventory Management

Inventory must be maintained per warehouse and product.

```
Product: MacBook Pro

Kolkata Warehouse → 15
Mumbai Warehouse  → 8
Delhi Warehouse   → 21
```

Suggested structure:

```
inventory
  id
  organizationId
  warehouseId
  productId
  availableQuantity
  reservedQuantity
  version
  updatedAt
```

Operations: add inventory, remove inventory, transfer inventory, reserve inventory, release inventory, and inventory history. The system should prevent inventory from becoming negative.

7. Order Management

Users should be able to create orders containing multiple products.

```
{
  "warehouseId": "WH001",
  "customerId": "CUS001",
  "items": [
    { "productId": "P001", "quantity": 2 },
    { "productId": "P002", "quantity": 1 }
  ]
}
```

Order statuses: PENDING, CONFIRMED, PROCESSING, COMPLETED, CANCELLED.

Creating an order should:

```
Request
  |
  ▼
Validate User → Validate Organization → Validate Products
  |
  ▼
Check Inventory → Reserve Inventory → Create Order
  |
  ▼
Generate Audit Event → Publish Notification Event
```

8. Concurrency Handling (Mandatory)

Consider the following:

```
Available stock = 1

User A orders 1
User B orders 1

Both requests arrive at the same time.
```

The application must prevent both requests from successfully purchasing the same inventory. Implement or document a suitable strategy such as optimistic locking, pessimistic locking, atomic database updates, distributed locking, or transaction isolation. **You must justify the chosen approach.**

9. Real-Time Notification System

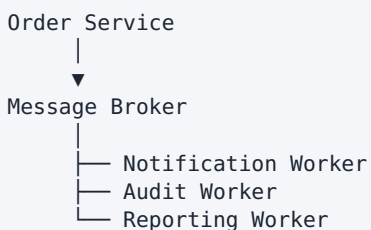
When important operations occur, connected users should receive notifications, for example: Order Created, Order Cancelled, Low Inventory, Inventory Updated, Inventory Transfer Completed.

Suggested technologies: WebSocket, Socket.IO, or Server-Sent Events.

```
{
  "event": "INVENTORY_LOW",
  "productId": "P001",
  "warehouseId": "WH001",
  "remainingQuantity": 3
}
```

10. Event-Driven Processing

At least one asynchronous workflow should be implemented.



Possible technologies: Kafka, RabbitMQ, Redis Streams, or AWS SQS. You may simplify infrastructure if required, but should explain how the production version would work.

11. Audit Logging

Important operations should generate an immutable audit trail.

```
{
  "userId": "USER123",
  "organizationId": "ORG001",
  "action": "INVENTORY_UPDATED",
  "entity": "Inventory",
  "entityId": "INV001",
  "oldValue": { "quantity": 20 },
  "newValue": { "quantity": 15 },
  "timestamp": "2026-09-18T10:30:00Z"
}
```

Track operations including login, user creation, product changes, inventory changes, order status changes, and permission changes.

12. Dashboard

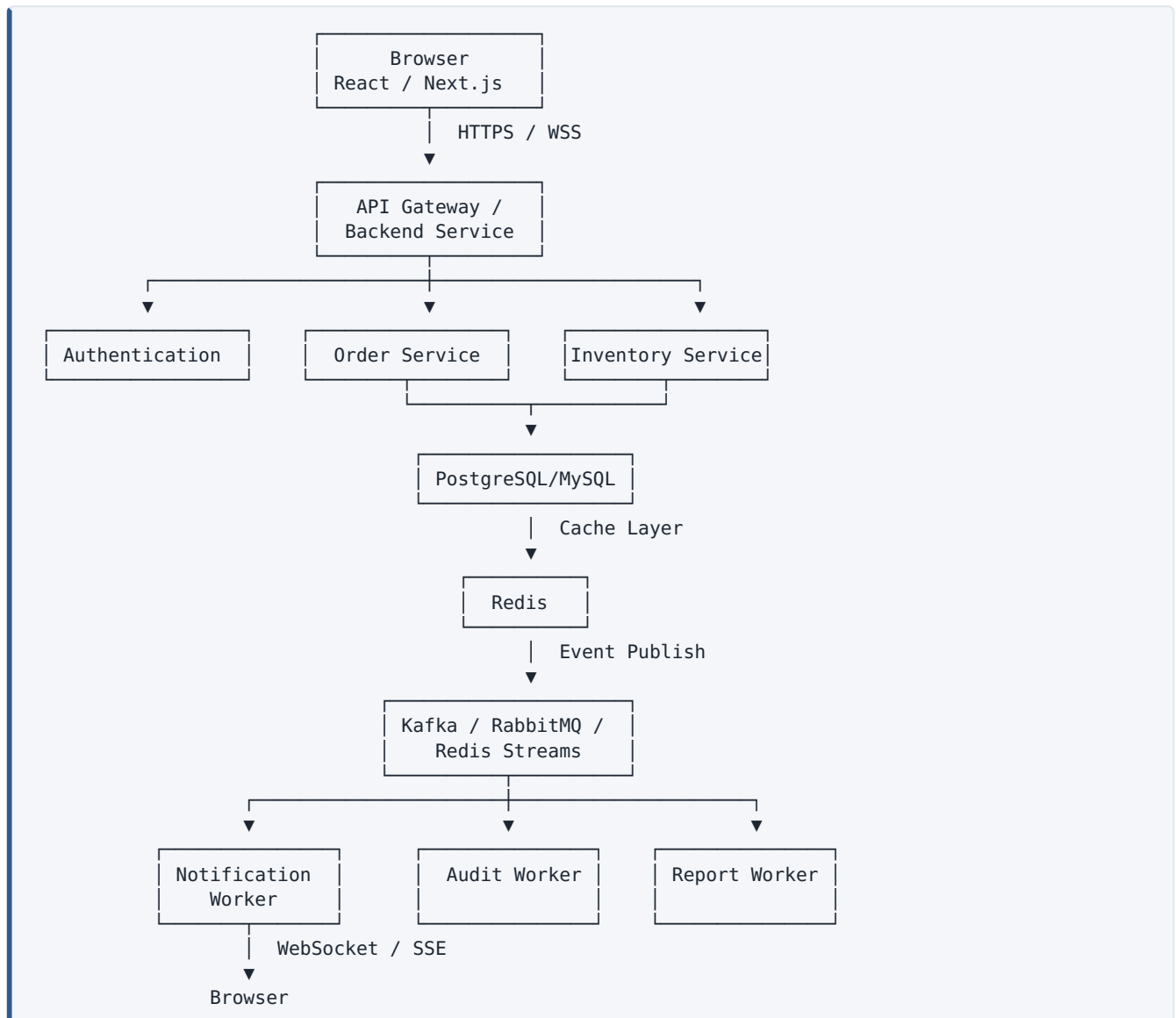
Create an administration dashboard. Minimum widgets:

Total Products	Total Warehouses	Available Inventory
Pending Orders	Completed Orders	Low Stock Products
Recent Orders		

Include pagination, search, loading state, error state, and empty state. Responsive design should be considered.

13. High-Level Architecture

Recommended architecture:



A modular monolith is completely acceptable. **Do not create microservices merely for the sake of using microservices.** Architectural decisions should be justified.

14. Suggested Backend Structure

```
src/
├── auth/
├── roles/
├── inventory/
├── audit/
├── common/
│   ├── guards/
│   ├── validators/
├── infrastructure/
│   ├── database/
│   └── websocket/
└── app

├── organizations/
├── products/
├── orders/
├── middleware/
├── utilities/
├── redis/
├── users/
├── warehouses/
├── notifications/
├── exceptions/
└── queue/
```

Architecture could follow: Controller → Application / Service Layer → Domain Layer → Repository → Database. You may use Clean Architecture, Hexagonal Architecture, Domain-Driven Design, or a Modular architecture — the architectural choice must be explained.

15. Suggested Frontend Structure

```
src/
├── app/
├── modules/
│   ├── auth/
│   ├── orders/
├── hooks/
├── types/
├── components/
├── products/
├── dashboard/
├── services/
├── utils/
├── features/
├── inventory/
└── store/
```

The frontend should demonstrate component reuse, separation of server/client state, error handling, form validation, authentication handling, and authorization handling.

16. Database Design

You should provide an ER diagram. Minimum entities and key relationships:

```
Organization 1 — N Users
Organization 1 — N Warehouses
Organization 1 — N Products
Organization 1 — N Customers
Organization 1 — N Orders
Warehouse 1 — N Inventory
Product 1 — N Inventory
Order 1 — N OrderItems
Product 1 — N OrderItems
```

Identify the required primary keys, foreign keys, unique constraints, indexes, and composite indexes.

17. API Standards

REST or GraphQL may be used. For REST, recommended endpoints include:

```
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout

GET /api/v1/products
POST /api/v1/products
GET /api/v1/products/:id
PATCH /api/v1/products/:id

GET /api/v1/warehouses
POST /api/v1/warehouses

GET /api/v1/inventory
POST /api/v1/inventory/adjust
POST /api/v1/inventory/transfer

GET /api/v1/orders
POST /api/v1/orders
GET /api/v1/orders/:id
PATCH /api/v1/orders/:id/status

GET /api/v1/audit-logs
```

A consistent API response format should be used.

```
Success:
{ "success": true, "data": {}, "meta": {} }

Failure:
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_INVENTORY",
    "message": "Requested quantity is unavailable"
  }
}
```

18. API Requirements

Implement pagination, filtering, sorting, validation, global exception handling, appropriate HTTP status codes, and API versioning. Example:

```
GET /api/v1/products?page=1&limit=20&search=laptop&sort=createdAt:desc
```

19. Caching

Use Redis or another caching mechanism for at least one appropriate use case — for example frequently accessed product information, dashboard metrics, user session information, or rate limiting. You must explain your cache key strategy, TTL, cache invalidation, and potential stale-data scenarios.

20. Security Requirements

Your implementation should consider: password hashing, JWT validation, refresh token security, RBAC, tenant isolation, request validation, SQL/NoSQL injection, XSS, CORS, rate limiting, sensitive information handling, environment variables, and secure HTTP headers. **Secrets must not be committed to Git.**

21. Logging & Observability

Structured application logging should be implemented.

```
{
  "timestamp": "2026-09-18T10:30:00Z",
  "level": "INFO",
  "requestId": "REQ-123",
  "organizationId": "ORG-1",
  "userId": "USER-1",
  "method": "POST",
  "path": "/api/v1/orders",
  "duration": 143
}
```

Bonus: request correlation IDs, Prometheus metrics, Grafana, OpenTelemetry, health checks, and distributed tracing. Required endpoints could include `/health` and `/ready`.

22. Testing (Mandatory)

Unit Tests

Test business logic such as inventory reservation, inventory release, order calculations, authorization, and tenant isolation.

Integration Tests

POST /orders → Inventory validated → Inventory reserved → Order created

Critical Concurrency Test

Demonstrate that multiple concurrent requests cannot oversell inventory (e.g. 100 simultaneous requests against an available inventory of 10 must result in a maximum of 10 successful purchases).

Bonus: end-to-end tests, frontend tests, load tests.

23. Docker

The complete project should run through Docker via `docker compose up`. Suggested containers:

frontend backend postgres redis message-broker

24. CI/CD

Provide a basic CI pipeline. Example GitHub Actions flow:

Push / Pull Request → Install Dependencies → Lint → Unit Tests
→ Integration Tests → Build → Docker Image

Actual cloud deployment is optional.

25. Mandatory Documentation

The repository must contain the following files:

README.md	ARCHITECTURE.md	API.md
DATABASE.md	SECURITY.md	SETUP.md

The README should contain: project overview, technology stack, installation, environment configuration, running locally, running through Docker, running tests, architecture overview, API documentation, known limitations, and future improvements.

26. Architecture Decision Records (ADRs)

Document important decisions. Example:

ADR-001 – PostgreSQL vs MongoDB
Decision: PostgreSQL
Reason: Orders and inventory require strong transactional consistency.
Trade-off: Horizontal scaling requires additional planning.

At least **3 ADRs** are recommended. Possible subjects: database selection, monolith vs microservices, queue technology, caching strategy, concurrency strategy, authentication mechanism.

27. Failure Scenarios

Explain how the application behaves when:

Database unavailable Redis unavailable
Message broker unavailable WebSocket disconnected
Duplicate order request Inventory changes during checkout
Two simultaneous inventory updates occur

Bonus: use an idempotency key to prevent duplicate order creation, e.g. Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000 .

28. Scalability Discussion

Explain how the architecture would evolve from 1,000 users to 1,000,000 users. Topics should include horizontal scaling, load balancing, database replicas, database partitioning, caching, CDN, message queues, async processing, stateless services, connection pooling, WebSocket scaling, observability, and rate limiting.

Implementation of all scaling mechanisms is not required — the reasoning is more important.

29. Expected Technology Stack

Technology choice is open. Suggested options:

Layer	Options
Backend	Node.js + NestJS · Java + Spring Boot · .NET · Go
Frontend	React · Next.js · Angular · Vue
Database	PostgreSQL · MySQL
Infrastructure	Redis · Kafka / RabbitMQ · Docker · Docker Compose
Testing	Jest · JUnit · Vitest · Playwright · Cypress

30. Submission Requirements

Please submit: a Git repository, README, architecture documentation, database diagram, API documentation, Docker configuration, source code, tests, and a CI pipeline.

Optional: deployed URL, Swagger/OpenAPI URL, Postman collection, demo video.

31. Evaluation Criteria

Area	Weight
Architecture & System Design	20%
Backend Engineering	20%
Concurrency & Data Integrity	15%
Database Design	10%
Security	10%
Real-Time & Event-Driven Design	10%
Testing	10%
Documentation & ADRs	5%
Total	100%

If any requirement is unclear, state your assumptions in the README and proceed. We look forward to reviewing your submission. — Uphead Management Consulting